

Asset Pack User-Guide

<https://slipperhat.itch.io/>

Last updated: 15/09/2024 – v1.01

Tested with:

Unity - 2022.3.46f1

Godot - 4.3

Unreal Engine – 5.4.4

Contents (click to navigate)

TLDR.....	2
FAQ.....	2
glTF 2.0 (.glb file format) – RECOMMENDED	3
Unity.....	3
Installing Unity glTFast.....	3
Importing glTF 2.0 into Unity.....	4
Godot.....	4
Importing glTF 2.0 into Godot.....	4
Unreal.....	5
Importing glTF 2.0 into Unreal.....	5
Manual Material Creation.....	6
Unity URP workflow.....	6
Godot ORM workflow.....	7
Unreal.....	8

TLDR

You do not need to use every file in the asset folders, there are 3 importing formats to choose from:

1. glTF 2.0 (.glb) is a single import options, with the materials pre-applied to the mesh.
2. OBJ PBR workflow. Includes mesh, albedo, normal and packed OcclusionRoughnessMetallic
3. Unity URP Glossiness workflow. Includes albedo, normal, AO and packed MetallicSmoothness

GLTF 2.0 (.glb) is recommended for beginners, it is the quickest and easiest solution, leaving no room for materials to be applied incorrectly.

Other workflows allow for more customisation and manually tuning variables in the material shader.

Your mileage may vary with the Unity URP workflow and you will likely have to manually tune the glossiness to your liking.

FAQ

1. Why are the backs of the models not textured / blank?

These assets were originally made for Itch.io jam games, where filesize becomes a consideration in 3D. They are intended for flat surfaces such as walls, ceilings and floors where only 1 face is visible to the player. All available UV resolution is used in the visible faces to maximise resolution in a 1k texture, keeping the filesize and memory footprint small, while having nice resolution.

2. Can I use this pack in my commercial game / project?

Yes. This asset pack is licensed under CC0. You can use these assets in any commercial or personal project without attribution. The LICENSE.txt file has more information.

4. Can I make changes to these assets?

Yes, you can edit these assets as you wish. For example, you could import the albedo into Photoshop / GIMP and change the colours to fit your project. You are permitted to adapt & build upon these assets in anyway you wish.

5. I'm stuck, can I ask you a question?

Yes, please make sure your question isn't answered in this document. If you're still stuck you can message me on Discord at slippers9631 and I'll do my best to help.

6. Can I have a .blend / .spp / .mtl / .fbx / other file?

I don't have any intentions currently to supply other file formats, as I would have to go and recreate the textures in other tools which would require quite a lot of work. If there is demand for this I could possibly do it but it would likely be as part of a paid pack beyond the free version.

You are free to import the assets into Blender to export them in another format suitable for your uses.

glTF 2.0 (.glb file format) – **RECOMMENDED**

It is strongly recommended to use the .glb files in most use cases. The mesh and textures are packed into a single, performant binary file. All major engines have support for this format and it ensures that the assets are imported into your project exactly as they were designed.

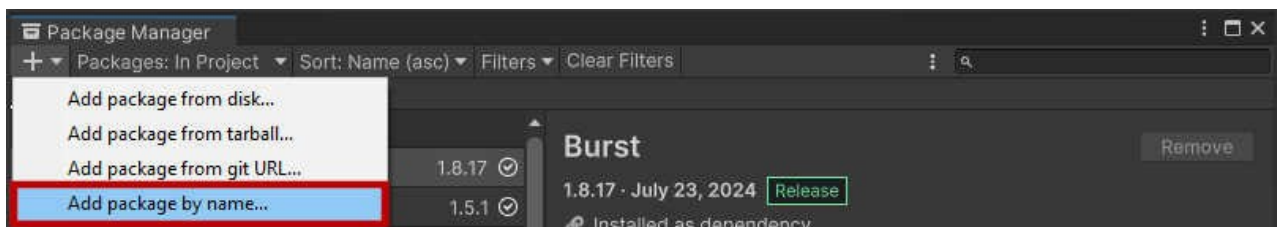
Unity

Unity supports glTF 2.0 through the Unity glTFast plugin, this is a plugin made by Unity themselves and allows for an easy and fast workflow for importing .glb files.

More information here: <https://docs.unity3d.com/Packages/com.unity.cloud.gltfast@6.8/manual/index.html>

Installing Unity glTFast

1. Open your Unity project and go to Windows > Package Manager.
2. Click the + in the top left of the Package Manager window and click on “Add package by name...”:

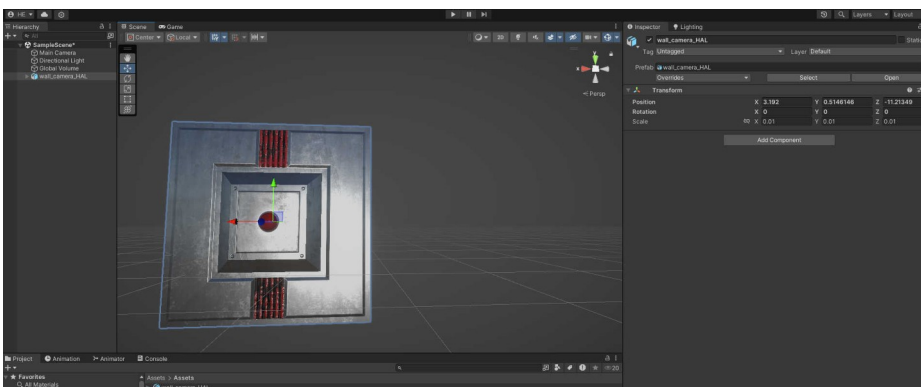


3. In the Name field, enter **com.unity.cloud.gltfast**
4. Click Add.
5. You should see the plugin has successfully installed:



Importing glTF 2.0 into Unity

1. Go to the folder 'ASSET_NAME' > GLB and import the enclosed file into a folder in your Unity project.
2. Simply drag the asset into the scene window and the mesh and textures will be imported and ready to use:



Godot

Godot natively supports glTF 2.0 and it is the format recommended in the documentation:

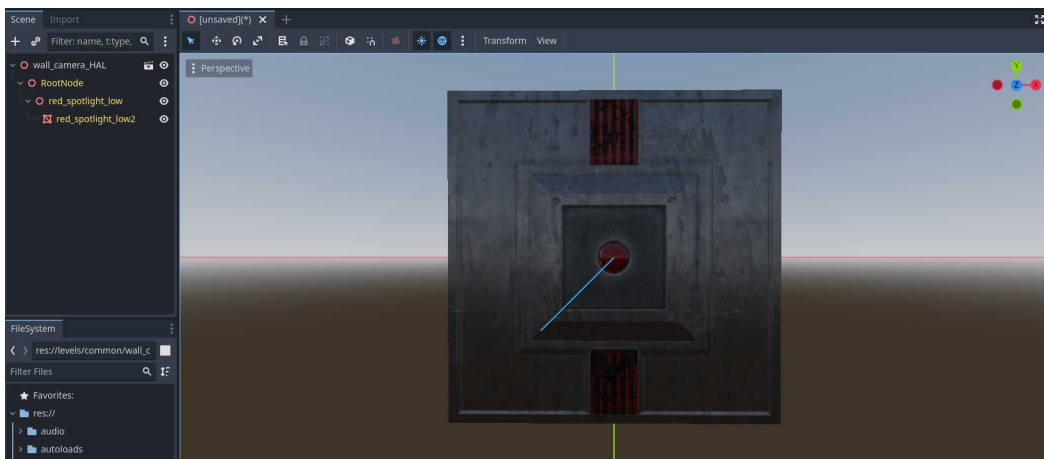
https://docs.godotengine.org/en/stable/tutorials/assets_pipeline/importing_3d_scenes/available_formats.html

Importing glTF 2.0 into Godot

1. Go to the folder ASSET_NAME > GLB and import the enclosed file into a folder in your Godot project.
2. Godot will create a new scene file for your asset:



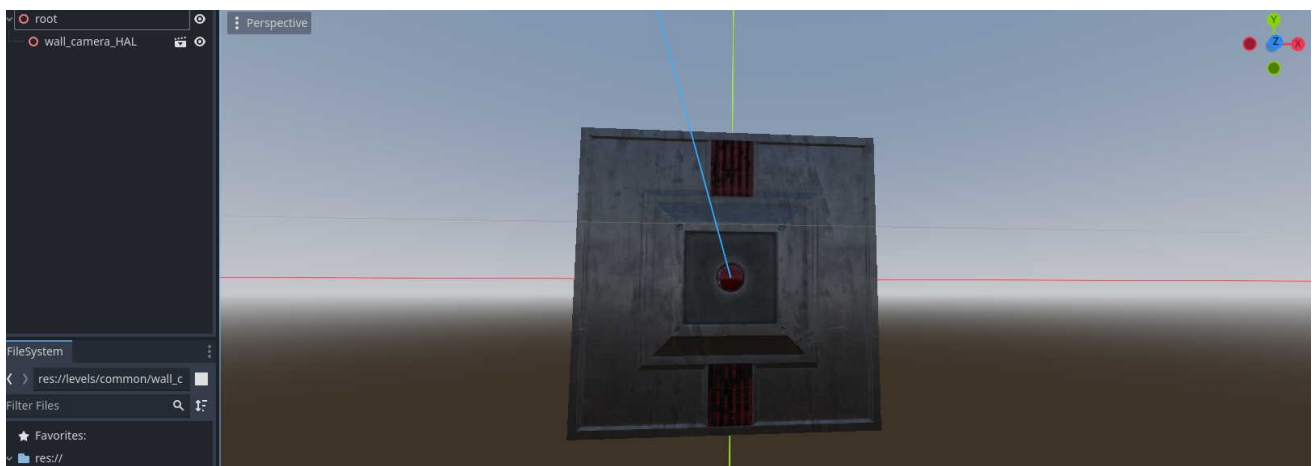
3. Right click on the scene and click “New Inherited Scene” which will open a new tab:



4. Save this .tscn file and drag this into your game scene every time you wish to use this asset.

N.B. these instantiations will inherit the data from the original scene, and will only duplicate data that is changed between instantiations.

Do not drag the .glb file into the project every time, make an inherited scene and use that.



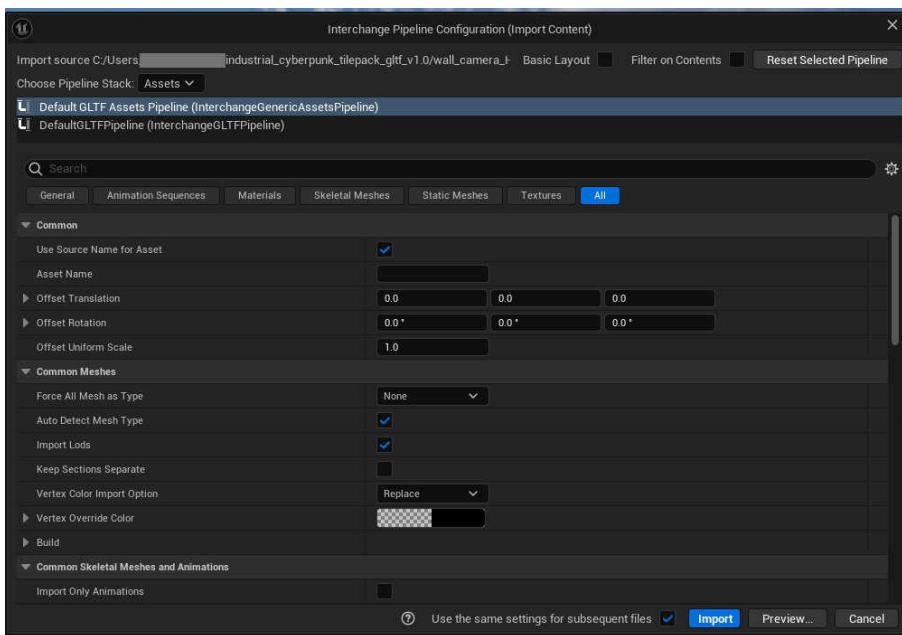
Unreal

Unreal Engine natively supports glTF 2.0, more information can be found here:

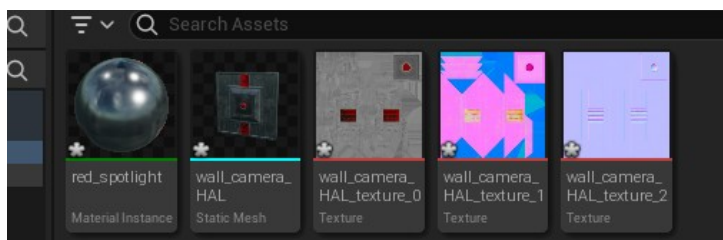
<https://dev.epicgames.com/documentation/en-us/unreal-engine/gltf-file-format-support-in-unreal-engine>

Importing glTF 2.0 into Unreal

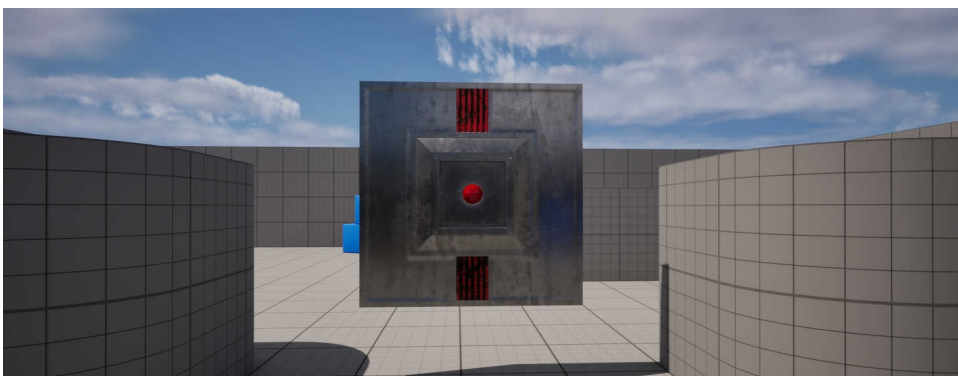
1. Go to the folder ASSET_NAME > GLB and import the enclosed file into a folder in your Unreal project.
2. Unreal will open the Interchange Pipeline Configuration:



3. If you are not familiar with the import process, you can safely click import with the default settings and Unreal will create your mesh and materials:



5. Drag and drop the 'Static Mesh' into your scene:



Manual Material Creation

Manually creating the materials from the supplied textures is not recommended for beginners. There are a number of ways to achieve the same effects in each engine and below are just examples for Unity, Godot and Unreal. There are specific workflows that can work better for different types of material. Please ensure you save and reuse materials, don't recreate materials you have already made.

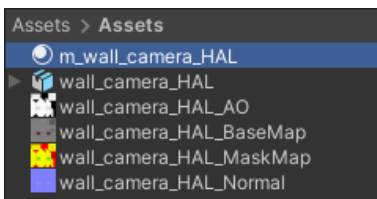
Unity URP workflow

1. Go to the folder 'ASSET_NAME' > OBJ and import the enclosed file into a folder in your Unity project. This is the base mesh.

2. Go into the 'OMS Unity Textures' folder and import the enclosed textures into your Unity project:

N.B. There will usually be 4 files, but sometimes the AO is omitted, this is normal as it's not always needed. You can omit AO for all your models if you wish not to use it.

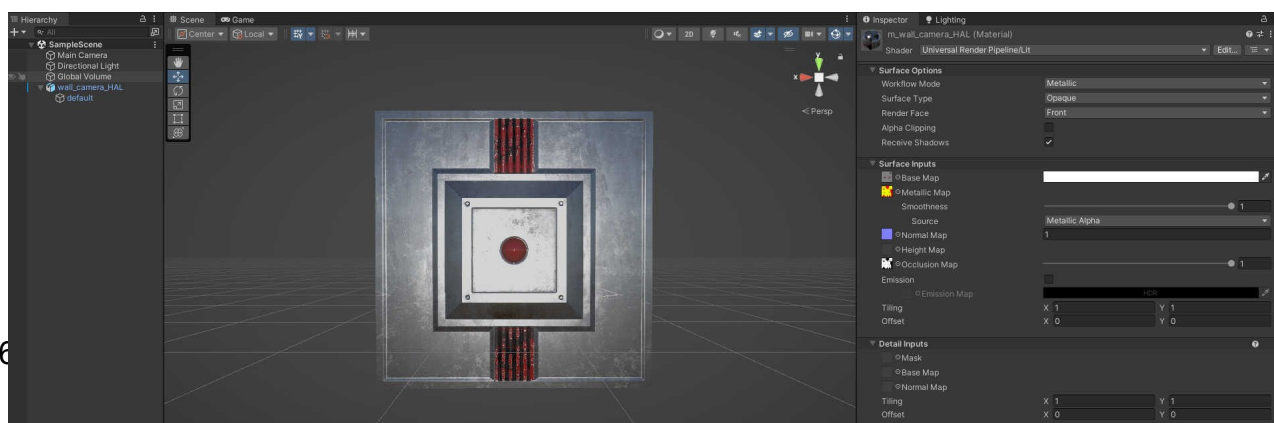
3. Right click in your asset folder and click 'create' > 'material' and give it a name:



4. Left click the 'Normal' texture and in the inspector on the right change the 'Texture Type' from 'Default' to 'Normal Map'. Click 'Apply'.

5. Click back on the material, and you can start to drag and drop the materials onto the 'Surface Inputs' in the inspector on the right:

1. Drag 'BaseMap' into the Base Map input.
2. Drag 'Normal' into the Normal Map input.
3. Drag 'AO' (if included) into the Occlusion Map input.
4. Drag 'MaskMap' into the Metallic Map input. You may have to adjust the Smoothness slider to your liking, for metallic / shiny objects you may have to increase it all the way to 1.
5. Drag the Mesh into the scene, and drag the material onto the mesh.



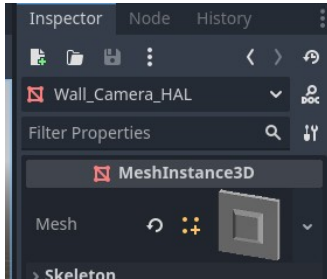
Godot ORM workflow

1. Go to the folder 'ASSET_NAME' > OBJ and import the enclosed file into a folder in your Godot project. This is the base mesh.

2. Go into the 'ORM Textures' folder and import the enclosed textures into your Godot project:

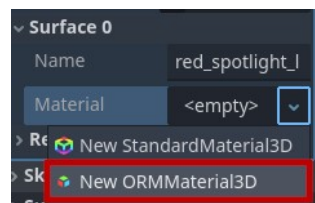
3. Within your scene create a new MeshInstance3D node and give it a name.

4. Drag and drop the imported .obj file into the Mesh property in the Inspector on the right for the MeshInstance3D:



5. Right click your Mesh in the inspector, and click 'Make Unique' towards the bottom.

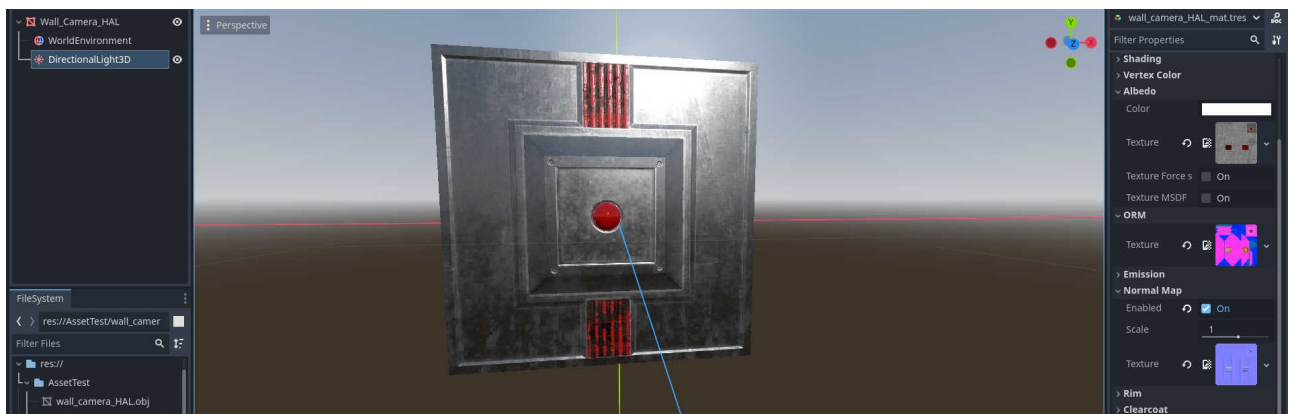
6. Left click the Mesh in the inspector and under 'Surface 0' click the 'Material' drop down arrow and select 'New ORMMaterial3D':



7. Right click the new material, click 'Save As...', give it a name and save it in your desired folder (recommended to save it in the same folder as your mesh and textures).

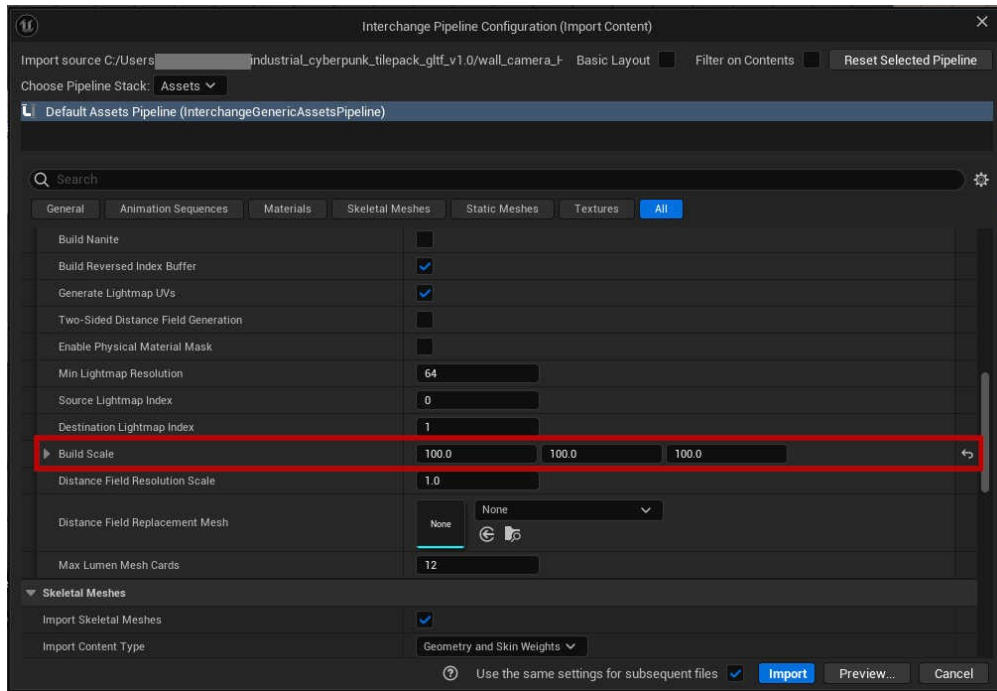
8. Double click the material, and you can start to drag and drop the materials into the inspector on the right:

1. Drag 'baseColor' into 'Albedo' > 'Texture'.
2. Click the 'Normal Map' dropdown and left click the 'Enabled' checkbox
3. Drag 'normal' into 'Normal Map' > 'Texture'
4. Drag 'occlusionRoughnessMetallic' into 'ORM' > 'Texture'



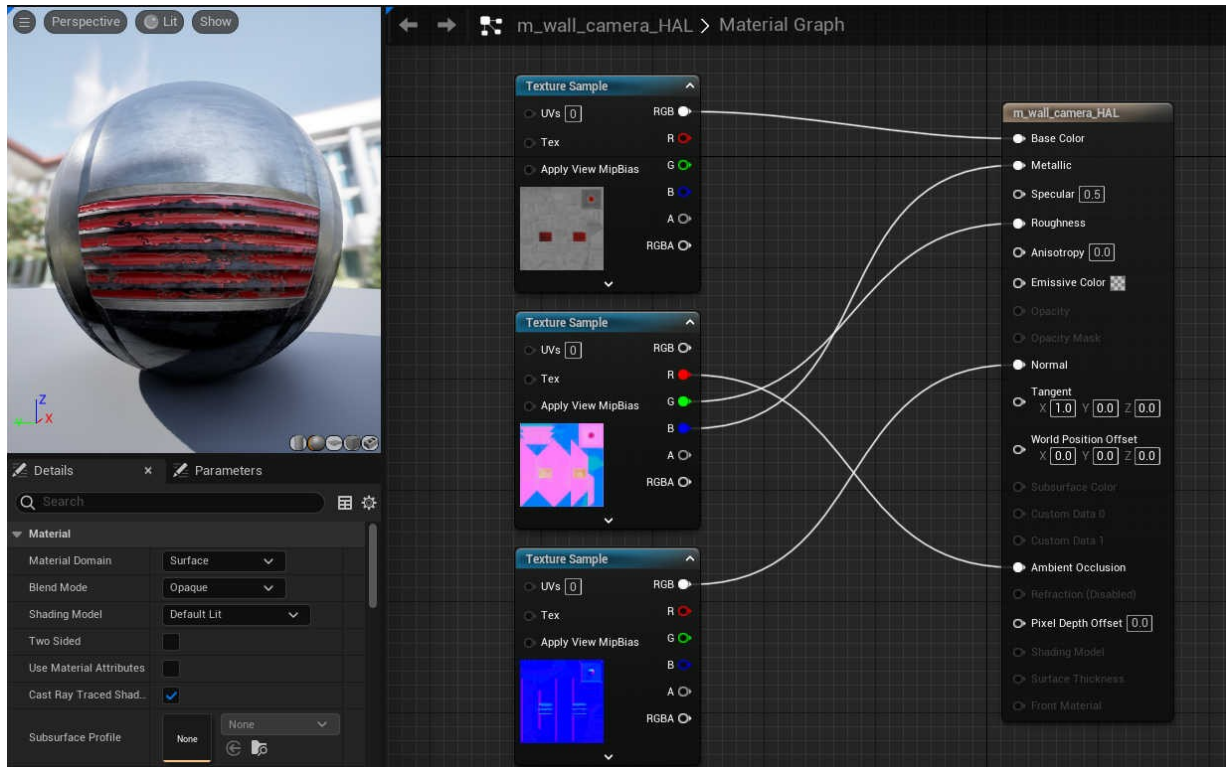
Unreal

1. Go to the folder 'ASSET_NAME' > OBJ and import the enclosed file into a folder in your Unreal project. This is your base mesh.
2. Unreal will open the Interchange Pipeline Configuration:
3. Under the 'Build Scale' option, change each input from '1' to '100'.



N.B. You can alternatively scale all of the meshes in Blender. By default the unit size in blender is metres, and the default in Unreal is centimetres. I have supplied the meshes in metres for this pack.

3. If you are not familiar with the import process, you can safely click import now.
4. Go into the 'ORM Textures' folder and import the enclosed textures into your Unreal project.
5. Double click the 'occlusionRoughnessMetallic' texture and in the 'Details' panel on the right, under 'Texture', uncheck the sRGB box and save.
6. Open the your new material and drag in the 'baseColor', 'normal' and 'occlusionRoughnessMetallic' textures from the content drawer into the Material Graph.
 1. Connect the 'baseColor' Texture Sample to the 'Base Color' input
 2. Connect the 'normal' Texture Sample to the 'Normal' input
 3. Click the drop down on the 'occlusionRoughnessMetallic' Texture sample and set the Sampler Type to 'Linear Color'. Connect the R output to 'Ambient Occlusion'. Connect the G output to 'Roughness'. Connect the B output to 'Metallic':



7. Click 'Apply'. Drag the mesh into your scene and apply the material.

N.B Some of the static meshes were flickering in my testing. This was solved by increasing the Positive Bounds Extention under General > Advanced in the static mesh settings. Alternatively use the glTF 2.0 workflow.

